

Tabungan Haji API

REST API untuk produk **Tabungan Haji** — registrasi nasabah, pembukaan rekening, pencatatan transaksi (setoran/penarikan & QRIS), estimasi keberangkatan haji, serta export laporan transaksi bulanan. Dibangun dengan **Node.js + TypeScript**, **Express 5**, **Prisma 6**, dan **PostgreSQL**.

Daftar Isi

- Fitur Utama
- Tech Stack
- Arsitektur
- Struktur Proyek
- Prasyarat
- Instalasi & Setup
- Menjalankan Aplikasi
- Konsep Inti
 - Format Response
 - Tipe Data BigInt
 - Autentikasi (JWT)
 - Daftar Kode Error
- Model Data
- Referensi API
 - Health Check
 - Modul Nasabah
 - Modul User & Autentikasi
 - Modul Tabungan
 - Modul Transaksi
- Estimasi Keberangkatan Haji
- Export Laporan Bulanan (CSV)
- Pengujian dengan Postman
- Database & Migrasi
- Keamanan
- Catatan & Batasan
- Regenerasi Dokumentasi PDF
- Lisensi

Fitur Utama

Domain	Kemampuan
Nasabah	CRUD data calon jamaah (NIK, nama, email, nomor HP)
User & Auth	Registrasi akun login per nasabah, login (JWT), logout (revoke token), CRUD user
Tabungan	Buka rekening, lihat saldo, ubah status (AKTIF/BLOKIR/TUTUP), hapus rekening
Transaksi	Setoran & penarikan atomik, setoran via QRIS, riwayat transaksi
Estimasi Haji	Hitung kelayakan porsi, estimasi tahun keberangkatan, dan sisa kuota dari saldo
Laporan	Export laporan transaksi bulanan dalam format CSV

Tech Stack

Komponen	Teknologi	Versi (package.json)
Runtime	Node.js	≥ 18 (diuji pada v24)
Bahasa	TypeScript	^6.0.3
Web framework	Express	^5.2.1
ORM	Prisma + @prisma/client	^6.19.3
Database	PostgreSQL	—
Validasi	Zod	^4.4.3
Hashing password	bcrypt	^6.0.0 (12 salt rounds)
Autentikasi	jsonwebtoken	^9.0.3
Keamanan HTTP	helmet, cors	^8.2.0 / ^2.8.6
Dev tooling	ts-node, nodemon	—

Arsitektur

Setiap domain dipecah menjadi modul mandiri dengan empat lapisan yang konsisten:

```
route      → mendefinisikan endpoint + memasang middleware (authenticate)
└─ controller → validasi input (Zod), mapping error → HTTP, format response
    └─ service   → logika bisnis + akses database (Prisma)
        └─ schema    → skema validasi Zod + tipe TypeScript turunannya
```

Aliran sebuah request:

```
Client → Express → helmet/cors/json → [authenticate?] → route → controller → service → Prisma → PostgreSQL
```

Prinsip pemisahan tanggung jawab:

- **Controller** tidak menyentuh database; ia hanya memvalidasi, memanggil service, dan menerjemahkan hasil/exception menjadi response HTTP.
- **Service** tidak tahu soal HTTP; ia mengembalikan data atau melempar *business error* yang khusus (mis. `TransaksiBusinessError`).
- **Schema** adalah satu-satunya sumber kebenaran untuk aturan validasi dan tipe input.

Struktur Proyek

```
tabungan-haji-online/
├── prisma/
│   ├── migrations/          # Riwayat migrasi (init, add_user_model, ...)
│   └── schema.prisma        # Definisi model & relasi
├── postman/
│   ├── tabungan-haji-api.postman_collection.json
│   └── tabungan-haji-api.postman_environment.json
├── docs/
│   ├── API.md               # Dokumentasi modul Nasabah (legacy)
│   ├── tabungan-transaksi.md # Dokumentasi modul Tabungan & Transaksi
│   └── Tabungan-Haji-API.pdf # Versi PDF dari README ini
├── scripts/
│   └── build-pdf.mjs         # Generator PDF (README → PDF via Chrome)
├── src/
│   ├── index.ts             # Entry point Express + serializer BigInt
│   ├── lib/
│   │   ├── prisma.ts        # Singleton PrismaClient
│   │   ├── jwt.ts           # Sign & verify JWT
│   │   └── token-denylist.ts # Denylist token (in-memory) untuk logout
│   ├── middleware/
│   │   └── auth.ts          # Middleware authenticate (Bearer token)
│   └── modules/
│       ├── nasabah/ (route, controller, service, schema)
│       ├── user/     (route, controller, service, schema)
│       ├── tabungan/ (route, controller, service, schema)
│       └── transaksi/ (route, controller, service, schema)
├── .env                  # Variabel lingkungan (tidak di-commit)
├── prisma.config.ts      # Konfigurasi Prisma CLI
├── tsconfig.json
└── package.json
```

Prasyarat

- **Node.js** ≥ 18 (disarankan LTS terbaru).
- **PostgreSQL** ≥ 13 yang berjalan dan dapat diakses.
- **npm** (terpasang bersama Node.js).

Instalasi & Setup

1. Install dependency

```
npm install
```

2. Konfigurasi variabel lingkungan

Buat file `.env` di root proyek:

```
# Koneksi PostgreSQL
DATABASE_URL="postgresql://USER:PASSWORD@HOST:5432/Tabungan_Haji?schema=public"

# Port HTTP (opsional, default 3000)
PORT=3000

# Rahasia & masa berlaku JWT
JWT_SECRET="ganti-dengan-string-acak-yang-panjang"
JWT_EXPIRES_IN="1h"
```

Variabel	Wajib	Default	Keterangan
DATABASE_URL	✓	—	Connection string PostgreSQL
PORT	✗	3000	Port server HTTP
JWT_SECRET	✓	—	Kunci penandatanganan JWT. Server gagal start bila kosong
JWT_EXPIRES_IN	✗	1h	Masa berlaku token (format <code>jsonwebtoken</code> , mis. <code>15m</code> , <code>1h</code> , <code>7d</code>)

`.env` sudah masuk `.gitignore` — jangan pernah commit kredensial.

3. Jalankan migrasi database

```
npx prisma migrate dev
```

Perintah ini membuat tabel sesuai `prisma/schema.prisma` dan men-generate Prisma Client.

Menjalankan Aplikasi

Perintah	Fungsi
<code>npm run dev</code>	Mode development — hot reload via <code>nodemon</code> + <code>ts-node</code>
<code>npm run build</code>	Kompilasi TypeScript ke <code>dist/</code>
<code>npm start</code>	Jalankan hasil build (<code>node dist/index.js</code>)

Setelah server hidup:

```
curl http://localhost:3000/health
# { "status": "ok", "service": "tabungan-haji-api", "timestamp": "..." }
```

Konsep Inti

Format Response

Sukses (objek tunggal):

```
{ "data": { ... }, "message": "..." }
```

Sukses (list):

```
{ "data": [ ... ], "total": 12 }
```

Error:

```
{ "error": "KODE_ERROR", "message": "Pesan untuk pengguna", "details": { ... } }
```

`details` hanya muncul pada error validasi Zod dan berisi hasil `z.flattenError ( formErrors + fieldErrors )`.

Tipe Data BigInt

Field nominal uang disimpan sebagai **BigInt** di PostgreSQL: `saldo` (Tabungan) dan `nominal / saldoSebelum / saldoSesudah` (Transaksi).

- Di **response JSON** nilai-nilai ini dikembalikan sebagai **string** — karena `BigInt` tidak punya representasi JSON native, `src/index.ts` memasang serializer global (`BigInt.prototype.toJSON`).
- Di **request** kirim sebagai **number** biasa; Zod akan memvalidasi sebagai integer.

Autentikasi (JWT)

Autentikasi memakai **Bearer token** JWT.

Alur:

1. Buat nasabah → `POST /api/v1/nasabah` (*publik*).
2. Buat akun login → `POST /api/v1/user` (*publik*).
3. Login → `POST /api/v1/user/login` (*publik*) → menerima `token` .
4. Sertakan header pada endpoint terproteksi:

```
Authorization: Bearer <token>
```

5. Logout → `POST /api/v1/user/logout` me-revoke token saat ini (masuk denylist sampai kedaluwarsa).

Isi payload token: `id` , `username` , `nasabahId` , objek `nasabah` (`id` , `nik` , `nama` , `email` , `nomorHp`), plus `jti` , `iat` , `exp` .

Endpoint publik (tanpa token): `GET /health` , `POST /api/v1/nasabah` , `POST /api/v1/user` , `POST /api/v1/user/login` .
Selebihnya wajib token.

Error autentikasi (semua HTTP 401):

error	Penyebab
UNAUTHORIZED	Header <code>Authorization</code> tidak ada / bukan <code>Bearer</code>
TOKEN_EXPIRED	Token kedaluwarsa
INVALID_TOKEN	Token tidak valid / tanda tangan salah
TOKEN_REVOKED	Token sudah di-logout (ada di denylist)

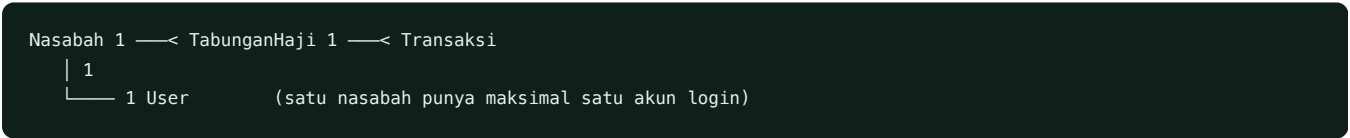
Daftar Kode Error

HTTP	error	Modul	Arti
400	VALIDATION_ERR	semua	Body/param/query gagal validasi Zod (lihat <code>details</code>)
400	INVALID_REFERENCE	tabungan	<code>nasabahId</code> (FK) tidak ditemukan saat buka rekening
401	UNAUTHORIZED / TOKEN_EXPIRED / INVALID_TOKEN / TOKEN_REVOKED	auth	Lihat tabel autentikasi
401	INVALID_CREDENTIALS	user	Username/password salah saat login
404	NOT_FOUND	semua	Resource tidak ada
404	TABUNGAN_NOT_FOUND	transaksi	Rekening target transaksi tidak ada
409	DUPLICATE_ENTRY	semua	Pelanggaran unique (NIK, email, username, nomorRekening, referensi)
409	TABUNGAN_NOT_ACTIVE	transaksi	Status rekening bukan <code>AKTIF</code>
422	NASABAH_NOT_FOUND	user	<code>nasabahId</code> (FK) tidak ada saat buat user
422	SALDO_INSUFFICIENT	transaksi	Saldo < nominal penarikan
422	SETORAN_BELOW_MIN	transaksi	Nominal setoran di bawah minimum (Rp 100.000)
500	(default)	semua	Error tak tertangani

Model Data

Skema lengkap ada di `prisma/schema.prisma`.

Relasi:



- Satu **Nasabah** dapat memiliki banyak **TabunganHaji** dan paling banyak satu **User**.
- Satu **TabunganHaji** dapat memiliki banyak **Transaksi**.

Nasabah (`nasabah`)

Field	Tipe	Keterangan
<code>id</code>	UUID	Primary key
<code>nik</code>	string(16)	Unique , tepat 16 digit
<code>nama</code>	string(100)	
<code>email</code>	string(150)	Unique
<code>nomorHp</code>	string(20)	Kolom <code>nomor_hp</code>
<code>createdAt</code> / <code>updatedAt</code>	DateTime	Otomatis

TabunganHaji (`tabungan_haji`)

Field	Tipe	Keterangan
id	UUID	Primary key
nasabahId	UUID	FK → nasabah.id
nomorRekening	string(20)	Unique , auto-generate bila kosong
saldo	BigInt	Default 0
status	string(20)	Default AKTIF (AKTIF BLOKIR TUTUP)
dibukaAt	DateTime	Default now()

Transaksi (transaksi)

Field	Tipe	Keterangan
id	UUID	Primary key
tabunganId	UUID	FK → tabungan_haji.id
jenis	string(20)	SETORAN PENARIKAN
nominal	BigInt	
saldoSebelum / saldoSesudah	BigInt	Snapshot saldo
referensi	string(50)	Unique , auto-generate bila kosong
metode	string(20)?	TUNAI TRANSFER DEBIT KARTU QRIS
waktu	DateTime	Default now()

User (users)

Field	Tipe	Keterangan
id	UUID	Primary key
nasabahId	UUID	Unique , FK → nasabah.id
username	string(70)	Unique
password	string(255)	Hash bcrypt, tidak pernah dikembalikan di response
createdAt / updatedAt	DateTime	Otomatis

Referensi API

- **Base URL:** http://localhost:3000
- **Prefix:** /api/v1
- **Content-Type:** application/json (kecuali export CSV)

Kolom **Auth** menandai apakah endpoint butuh Authorization: Bearer <token> .

Health Check

Method	Path	Auth
GET	/health	✗

```
{ "status": "ok", "service": "tabungan-haji-api", "timestamp": "2026-05-28T10:00:00.000Z" }
```

Modul Nasabah

Base path: /api/v1/nasabah

Method	Path	Auth	Keterangan
POST	/	✗	Registrasi nasabah baru
GET	/	✓	List semua nasabah (urut createdAt desc)
GET	/:id	✓	Detail nasabah
PATCH	/:id	✓	Update parsial (minimal 1 field)
DELETE	/:id	✓	Hapus nasabah (204)

Aturan validasi:

Field	Aturan
nik	Wajib, tepat 16 digit angka, unique
nama	Wajib, 3–100 karakter
email	Wajib, format email valid, ≤150 karakter, unique
nomorHp	Wajib, format 08xxxxxxxxxx (10–13 digit)

Contoh — Create (POST /api/v1/nasabah):

```
{  "nik": "3201020107950001",  "nama": "Andhini Wira Pratiwi",  "email": "andhini@example.com",  "nomorHp": "081234567890"}
```

Respons 201 :

```
{  "data": {    "id": "a4d9f2a7-1234-4abc-9876-0e1f2a3b4c5d",    "nik": "3201020107950001",    "nama": "Andhini Wira Pratiwi",    "email": "andhini@example.com",    "nomorHp": "081234567890",    "createdAt": "2026-05-28T10:00:00.000Z",    "updatedAt": "2026-05-28T10:00:00.000Z"  },  "message": "Nasabah berhasil dibuat"}
```

Error: 400 VALIDATION_ERR , 409 DUPLICATE_ENTRY (NIK/email), 404 NOT_FOUND (detail/update/delete).

Modul User & Autentikasi

Base path: /api/v1/user

Method	Path	Auth	Keterangan
POST	/	✗	Buat akun login untuk seorang nasabah
POST	/login	✗	Login, terbitkan JWT
POST	/logout	✓	Revoke token saat ini
GET	/	✓	List user (tanpa password)
GET	/:id	✓	Detail user
PATCH	/:id	✓	Update username / password
DELETE	/:id	✓	Hapus user (204)

Aturan validasi:

Field	Aturan
nasabahId	Wajib (create), UUID, unique (1 akun/nasabah)
username	4–70 karakter, hanya huruf/angka/._-, unique
password	8–72 karakter, di-hash bcrypt (12 rounds)

Contoh — Create User (POST /api/v1/user):

```
{ "nasabahId": "a4d9f2a7-...", "username": "andhini", "password": "rahasia123" }
```

Respons 201 mengembalikan data user **tanpa** field password .

Contoh — Login (POST /api/v1/user/login):

```
{ "username": "andhini", "password": "rahasia123" }
```

Respons 200 :

```
{
  "message": "Login berhasil",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "data": {
    "id": "...", "username": "andhini", "nasabahId": "a4d9f2a7-...",
    "nasabah": { "id": "...", "nik": "...", "nama": "...", "email": "...", "nomorHp": "..." }
  }
}
```

Logout (POST /api/v1/user/logout): sertakan Bearer token → { "message": "Logout berhasil" }. Setelah ini token tidak berlaku lagi.

Error: 400 VALIDATION_ERR , 401 INVALID_CREDENTIALS (username/password salah — pesan sama untuk mencegah *user enumeration*), 409 DUPLICATE_ENTRY (username), 422 NASABAH_NOT_FOUND (nasabahId tidak ada), 404 NOT_FOUND .

Modul Tabungan

Base path: /api/v1/tabungan — **semua endpoint butuh token.**

Method	Path	Auth	Keterangan
POST	/	✓	Buka rekening
GET	/	✓	List rekening (opsional ?nasabahId=)
GET	/:id/estimasi	✓	Estimasi keberangkatan haji & sisa kuota
GET	/:id	✓	Detail rekening + nasabah
PATCH	/:id	✓	Ubah status
DELETE	/:id	✓	Hapus rekening (204)

Aturan validasi (Create):

Field	Aturan
nasabahId	Wajib, UUID, harus merujuk nasabah yang ada
nomorRekening	Opsional, 10–20 digit. Kosong → auto-generate 13 digit
saldoAwal	Opsional, integer ≥ 0, default 0

Contoh — Buka rekening (POST /api/v1/tabungan):

```
{ "nasabahId": "a4d9f2a7-...", "saldoAwal": 500000 }
```

Respons 201 :

```
{
  "data": {
    "id": "b95bfc85-...",
    "nasabahId": "a4d9f2a7-...",
    "nomorRekening": "1716712345678",
    "saldo": "500000",
    "status": "AKTIF",
    "dibukaAt": "2026-05-28T10:00:00.000Z"
  },
  "message": "Tabungan berhasil dibuat"
}
```

Update status (PATCH /api/v1/tabungan/:id): body { "status": "BLOKIR" } — enum AKTIF | BLOKIR | TUTUP . Saldo **tidak** bisa diubah lewat endpoint ini; gunakan transaksi.

Error: 400 VALIDATION_ERR , 400 INVALID_REFERENCE (nasabahId tidak ada), 409 DUPLICATE_ENTRY (nomorRekening), 404 NOT_FOUND .

Detail endpoint **estimasi** ada di bagian [Estimasi Keberangkatan Haji](#).

Modul Transaksi

Base path: /api/v1/transaksi — **semua endpoint butuh token.**

Method	Path	Auth	Keterangan
POST	/	✓	Catat transaksi (SETORAN/PENARIKAN)
POST	/qris/setor	✓	Setoran via QRIS
GET	/	✓	List transaksi (opsional ?tabunganId=&jenis=)
GET	/laporan/bulanan	✓	Export laporan bulanan (CSV)
GET	/:id	✓	Detail transaksi + ringkasan tabungan

Operasi create bersifat atomik — saldo rekening dan record transaksi diperbarui dalam satu DB transaction (prisma.\$transaction).

Alur internal POST / :

1. Cari TabunganHaji by tabunganId → kalau tidak ada → 404 TABUNGAN_NOT_FOUND .
2. Cek status === "AKTIF" → kalau bukan → 409 TABUNGAN_NOT_ACTIVE .
3. Hitung saldoSesudah : SETORAN → saldo + nominal ; PENARIKAN → saldo - nominal .
4. Kalau saldoSesudah < 0 → 422 SALDO_INSUFFICIENT .
5. Update saldo + insert transaksi dalam transaction yang sama.

Aturan validasi (Create):

Field	Aturan
tabunganId	Wajib, UUID
jenis	Wajib, SETORAN PENARIKAN
nominal	Wajib, integer > 0. Untuk SETORAN minimal Rp 100.000
referensi	Opsional, 6–50 karakter, unique. Auto-generate STR-... / PNR-... / QRIS-...
metode	Opsional, TUNAI TRANSFER DEBIT KARTU QRIS

Contoh — Setoran (POST /api/v1/transaksi):

```
{ "tabunganId": "b95bfc85-...", "jenis": "SETORAN", "nominal": 250000, "metode": "TUNAI" }
```

Respons 201 :

```
{
  "data": {
    "id": "c3d4...",
    "tabunganId": "b95bfc85-...",
    "jenis": "SETORAN",
    "nominal": "250000",
    "saldoSebelum": "500000",
    "saldoSesudah": "750000",
    "referensi": "STR-1716712345678-0421",
    "metode": "TUNAI",
    "waktu": "2026-05-28T10:05:00.000Z"
  },
  "message": "Transaksi berhasil dicatat"
}
```

Setoran QRIS (POST /api/v1/transaksi/qris/setor): body { "tabunganId": "...", "nominal": 150000 } (minimal Rp 100.000). metode otomatis QRIS , referensi auto QRIS-... .

Error: 400 VALIDATION_ERR , 404 TABUNGAN_NOT_FOUND , 409 TABUNGAN_NOT_ACTIVE , 422 SALDO_INSUFFICIENT , 422 SETORAN_BELOW_MIN , 409 DUPLICATE_ENTRY (referensi).

Estimasi Keberangkatan Haji

GET /api/v1/tabungan/:id/estimasi — menghitung **estimasi keberangkatan haji** dan **sisa kuota** tahun berjalan berdasarkan **saldo** rekening. Read-only (tidak mengubah data).

Parameter perhitungan

Konstanta didefinisikan di `src/modules/tabungan/tabungan.schema.ts` dan mudah diubah sesuai kebijakan:

Konstanta	Default	Arti
SETORAN_AWAL_PORSI	25000000	Saldo minimum untuk memperoleh nomor porsi
BIAYA_PELUNASAN	56000000	Total biaya pelunasan (Bipih) per jamaah
KUOTA_HAJI_TAHUNAN	221000	Kuota nasional per tahun
MASA_TUNGGU_TAHUN	20	Rata-rata masa tunggu (tahun)

Logika

- Cari `TabunganHaji` by `id` → kalau tidak ada → `404 NOT_FOUND` .
- `eligiblePorsi = status === "AKTIF" dan saldo >= SETORAN_AWAL_PORSI` .
- Bila `eligible` → `estimasiTahunKeberangkatan = tahun sekarang + MASA_TUNGGU_TAHUN` ; bila tidak → `null` dan disertai `kekuranganSetoranAwal` .
- `porsiTerisi = jumlah rekening AKTIF yang saldonya ≥ setoran awal (proksi kuota terpakai); sisaKuota = max(0, KUOTA_HAJI_TAHUNAN - porsiTerisi)` .

Contoh respons (saldo memenuhi setoran awal)

```
{
  "data": {
    "tabunganId": "b95bfc85-...",
    "nomorRekening": "9892838190640",
    "status": "AKTIF",
    "nasabah": { "id": "2555...", "nik": "3201...", "nama": "Lisya Rasawari" },
    "saldo": "60000000",
    "setoranAwalPorsi": "25000000",
    "biayaPelunasan": "56000000",
    "eligiblePorsi": true,
    "lunas": true,
    "kekuranganSetoranAwal": "0",
    "kekuranganPelunasan": "0",
    "persenTerkumpul": 100,
    "masaTungguTahun": 20,
    "tahunPendaftaran": 2026,
    "estimasiTahunKeberangkatan": 2046,
    "kuotaTahunan": 221000,
    "tahunKuota": 2026,
    "porsiTerisi": 1,
    "sisaKuota": 220999,
    "keterangan": "Saldo memenuhi setoran awal porsi. Estimasi berangkat 2046 (masa tunggu 20 tahun).",
  },
  "message": "Estimasi keberangkatan haji berhasil dihitung"
}
```

Bila saldo belum cukup: `eligiblePorsi / lunas = false` , `tahunPendaftaran & estimasiTahunKeberangkatan = null` , dan `kekuranganSetoranAwal` berisi sisa yang harus disetor.

Error: `400 VALIDATION_ERR` (id bukan UUID), `404 NOT_FOUND` .

Export Laporan Bulanan (CSV)

GET `/api/v1/transaksi/laporan/bulanan` — mengunduh laporan transaksi satu bulan sebagai **file CSV** (bukan JSON). Rentang waktu `[awal bulan, awal bulan berikutnya)` dihitung berbasis **UTC**.

Query params

Field	Type	Wajib	Aturan
<code>tahun</code>	integer	✓	2000–2100
<code>bulan</code>	integer	✓	1–12
<code>tabunganId</code>	UUID	✗	Filter per rekening
<code>jenis</code>	enum	✗	<code>SETORAN</code> atau <code>PENARIKAN</code>

Respons

- Content-Type: `text/csv; charset=utf-8`
- Content-Disposition: `attachment; filename="laporan-transaksi-<tahun>-<bulan>.csv"` (mis. `laporan-transaksi-2026-05.csv`)
- Body diawali **BOM UTF-8** (agar Excel membaca karakter Indonesia dengan benar), baris dipisah **CRLF**.

Kolom (urut `waktu` **ascending**):

```
waktu,referensi,nomorRekening,namaNasabah,nik,jenis,metode,nominal,saldoSebelum,saldoSesudah
```

Bila tidak ada transaksi di bulan tersebut → tetap `200` dengan baris header saja.

Contoh

```
curl -OJ -H "Authorization: Bearer <token>" \
  "http://localhost:3000/api/v1/transaksi/laporan/bulanan?bulan=5&tahun=2026"

# Dengan filter:
curl -OJ -H "Authorization: Bearer <token>" \
  "http://localhost:3000/api/v1/transaksi/laporan/bulanan?bulan=5&tahun=2026&tabunganId=b95bfc85-...&jenis=SETORAN"
```

Contoh isi file:

```
waktu,referensi,nomorRekening,namaNasabah,nik,jenis,metode,nominal,saldoSebelum,saldoSesudah
2026-05-28T07:38:59.092Z,STR-1779953939092-0500,9953879406674,Widya Kharisma,3201020107950002,SETORAN,TUNAI,10000,10000,20000
```

Error: `400 VALIDATION_ERR` (bulan/tahun kosong atau di luar rentang), `401 UNAUTHORIZED`.

Pengujian dengan Postman

Import dua file dari folder `postman/` :

- `tabungan-haji-api.postman_collection.json`
- `tabungan-haji-api.postman_environment.json`

Collection memakai Bearer token `{{token}}` di level collection; endpoint publik di-set `No Auth`.

Urutan run yang disarankan:

- Health Check
- Nasabah → Create Nasabah (auto-set `nasabahId`)
- User → Create User lalu Login (auto-set `token`)

4. **Tabungan** → **Create Tabungan** (auto-set `tabunganId`)
5. **Transaksi** → **Create Setoran / Penarikan** (auto-set `transaksiId`)
6. **Tabungan** → **Estimasi Keberangkatan**
7. **Transaksi** → **Export Laporan Bulanan (CSV)** — `laporanBulan` / `laporanTahun` diisi otomatis ke bulan berjalan oleh pre-request script
8. **Negative Cases** (opsional)

Variabel `nasabahId`, `tabunganId`, `transaksiId`, `token` di-set otomatis oleh test script, sehingga request berikutnya langsung jalan.

Database & Migrasi

Perintah	Fungsi
<code>npx prisma migrate dev</code>	Terapkan migrasi + generate client (development)
<code>npx prisma migrate deploy</code>	Terapkan migrasi di production
<code>npx prisma generate</code>	Generate ulang Prisma Client
<code>npx prisma studio</code>	GUI inspeksi data

Migrasi yang tersedia: `..._init`, `..._add_user_model` (lihat [prisma/migrations/](#)).

Keamanan

- **Helmet** — header HTTP aman (CSP, HSTS, `X-Content-Type-Options`, dll.).
- **CORS** — diaktifkan untuk semua origin (sesuaikan untuk production).
- **Password hashing** — bcrypt dengan 12 salt rounds; password tidak pernah dikembalikan di response (Prisma `omit`).
- **JWT** — ditandatangani `JWT_SECRET`, masa berlaku `JWT_EXPIRES_IN`. Token membawa `jti` untuk revokasi.
- **Anti user-enumeration** — login selalu menjalankan `bcrypt.compare` (memakai hash dummy bila user tidak ada) dan mengembalikan pesan seragam `INVALID_CREDENTIALS`.
- **Denylist token** — logout memasukkan `jti` ke denylist sampai token kedaluwarsa.

Catatan & Batasan

- **Denylist token bersifat in-memory** (`src/lib/token-denylist.ts`) — cocok untuk satu instance dan **ter-reset saat server restart**. Untuk multi-instance/persisten, ganti dengan Redis atau penyimpanan eksternal.
- **Laporan bulanan memakai batas waktu UTC**, bukan zona waktu lokal (WIB). Pertimbangkan offset bila butuh batas bulan lokal.
- **Belum ada otorisasi berbasis peran (role)** — setiap pengguna terautentikasi dapat mengakses semua resource (perilaku konsisten dengan endpoint `findAll`). Tidak ada penyaringan kepemilikan per-nasabah.
- **Parameter estimasi haji** (setoran awal, biaya pelunasan, kuota, masa tunggu) adalah konstanta preset, bukan data resmi real-time — ubah di schema sesuai kebijakan.
- **sisaKuota** dihitung dari jumlah rekening aktif yang memenuhi setoran awal sebagai proksi, karena belum ada tabel porsi/daftar tunggu.

Regenerasi Dokumentasi PDF

Versi PDF dari README ini tersedia di [docs/Tabungan-Haji-API.pdf](#).

Untuk membuat ulang setelah mengubah README:

```
npm run docs:pdf
```

Script `scripts/build-pdf.mjs` mengubah `README.md` → HTML (via `marked`) → PDF menggunakan Google Chrome dalam mode headless. Prasyarat: Google Chrome terpasang (dipakai untuk rendering PDF).

Lisensi

ISC © andhini